SÉPTIMO LABORATORIO

En este laboratorio vamos a implementar algoritmos de optimización heurística. La optimización es una tarea intrínseca y fundamental en la IA (y en general en cualquier ciencia o ingeniería). Encontrar el máximo o el mínimo de una función es un problema clásico de las matemáticas que se resuelve analíticamente calculando los valores de la derivada de la función que como sabemos nos indican los valores extremos. El problema se complica cuando la función tiene varias variables, y entonces se recurre a cálculo de gradientes para obtener las trayectorias que van hacia el óptimo de la función.

Cuando no se conoce la formulación matemática de la función, como sucederá en este ejercicio, hay que recurrir a lo que se llaman metaheurísticas. Esto ocurre habitualmente en problemas de Machine Learning como el planteado en el laboratorio 4. Los métodos de optimización metaheurísticos se dividen fundamentalmente en dos: por un lado, los algoritmos de optimización que trabajan con una trayectoria única (también llamados metaheurísticas basadas en una sola solución o algoritmos de búsqueda local). Por otro lado, las metaheurísticas que trabajan con poblaciones.

En asignaturas del Grado de cursos posteriores se estudiarán las metaheurísticas de población. En este primer curso nos vamos a limitar a los métodos de optimización de trayectoria única que no son tan potentes, pero pueden dar soluciones aproximadas suficientemente buenas si las funciones a optimizar no son muy complejas mediante un tiempo de ejecución tasado.

En el fichero zip que se proporciona con esta práctica se encuentra un módulo Python *retos_optimizacion.py* y una carpeta *pyarmor_runtime_000000*.

- El módulo *retos_optimizacion.py* contiene la implementación de 4 clases denominadas Funcion_i (con i entre 1 y 4) en un formato codificado para que no se conozcan los valores iniciales de los constructores de sus objetos ni la operación matemática que realizan para evaluar (ver el código más abajo).
- La clave de la descodificación está en el archivo binario ya compilado *pyarmor_runtime.pyd* que se encuentra en la carpeta citada. Para que el proyecto funciones **ambas entidades (módulo y carpeta) deben estar en la misma carpeta a desarrollar.**

Se trata de implementar al menos cinco algoritmos de optimización heurística de trayectoria única para aproximar los mínimos de las funciones. Las cuatro funciones son de R^{10} en R y su valor mínimo global es 0 (por tanto, cuánto más bajo sea el valor de la función más cerca estará del óptimo).

Para usarlas solo debéis importar retos_optimizacion.py en vuestro código. Este sería un ejemplo básico de uso. Iniciamos valor a un valor de una lista con 10 ceros, construimos 4 objetos $f_i$que son usados para evaluar las funciones en el vector de ceros. Con este código puedes probar si te funciona el módulo proporcionado, aunque probablemente la IA genere más código del que se muestra:

```
import math
import numpy as np
import retos_optimizacion as reto

if __name__ == "__main__":
    valor=[0]*10
    f1 = reto.Funcion_1()
    f2 = reto.Funcion_2()
    f3 = reto.Funcion_3()
    f4 = reto.Funcion_4()
    print(f"Función 1 en cero vale: {f1.evaluar(valor):.4f}")
    print(f"Función 2 en cero vale: {f2.evaluar(valor):.4f}")
    print(f"Función 3 en cero vale: {f3.evaluar(valor):.4f}")
    print(f"Función 4 en cero vale: {f4.evaluar(valor):.4f}")
```

El contenido oculto del módulo *retos_optimización.py* (el fichero codificado) se colorea en verde o amarillo para que la función a optimizar sea desconocida. El código consiste en una clase padre *_BaseOpt* que tiene una propiedad que mide el número de invocaciones que se ha hecho a la función y otras dos funciones más. La primera "*reiniciar_contador*" inicia a cero el número de invocaciones o llamadas y la segunda "_preparar_x" recibe una lista de valores que representa el valor n-dimensional que es la entrada a la función y, tras comprobar que la dimensión de dicha lista es compatible con la dimensión con la que trabajan las funciones, devuelve un array de ***numpy*** a partir de los valores de la lista.

Como ya se ha dicho antes, las cuatro funciones a optimizar cuyo código no hemos desvelado para que no se puedan optimizar si no es con una formulación heurística tienen una inicialización y una función "evaluar" que recibe un valor de x mediante una lista n-dimensional y devuelve el valor de la función para ese x.

```
class _BaseOpt:
    def __init__(self, shift_vector):
        self._contador_evaluaciones = 0
        self._shift = np.asarray(shift_vector, dtype=float)
        self._dims = len(self._shift)

    @property
    def presupuesto_gastado(self):
        """Devuelve el número total de llamadas a la función."""
```

```
        return self._contador_evaluaciones

    def reiniciar_contador(self):
        """Pone a cero el contador de evaluaciones."""
        self._contador_evaluaciones = 0

    def _preparar_x(self, x):
        self._contador_evaluaciones += 1
        x = np.asarray(x)
        if len(x) != self._dims:
            raise ValueError(f"Se requieren exactamente {self._dims} dimensiones.")
        return x

class Funcion_1(_BaseOpt):
    def __init__(self):
        valor_inicial=????

    def evaluar(self, x):
        x = self._preparar_x(x)
        return ????

class Funcion_2(_BaseOpt):

    def __init__(self):
       valor_inicial= ????

    def evaluar(self, x):
        x = self._preparar_x(x)
        return ????

class Funcion_3(_BaseOpt):
    def __init__(self):
        valor_inicial= ????

    def evaluar(self, x):
        x = self._preparar_x(x)
        return ????

class Funcion_4(_BaseOpt):
    def __init__(self):
       valor_inicial= ????

    def evaluar(self, x):
        x = self._preparar_x(x)
        return ????
```

Se pide:

Implemente al menos cinco algoritmos de optimización de trayectoria única (consulte a su IA favorita) e implemente un framework de comparación para buscar el mínimo de cada función con cada modelo. El total de llamadas a cada función estará limitado a 10.000 y el intervalo de búsqueda para todas las funciones será el [-10,10].

Para la **reproducibilidad de los experimentos** sitúe la semilla de generación aleatoria a 1, mediante una sentencia como la siguiente al comienzo del programa principal:

np.random.seed(1)

Tenga en cuenta para la mejor resolución del problema que la mayoría de los métodos de optimización (tanto de trayectoria única como de población) tienen parámetros que hay que ajustar para conseguir la mejor búsqueda posible. Esta suele ser una paradoja habitual al usar estos métodos: para buscar el óptimo de una función se usa una búsqueda que depende de la optimización de unos parámetros que a su vez deberían ser buscados mediante una heurística ...

Para evitar este bucle sin fin lo habitual suele ser usar para los parámetros de la heurística una "grid-search" (consulte de nuevo con su IA favorita). Tenga en cuenta que el número de invocaciones a las cuatro funciones a optimizar está limitado a 10.000 para TODAS las invocaciones que se hagan, incluyendo la búsqueda de los parámetros de la búsqueda mediante una grid-search. No olvide indicar este detalle a la IA cuando genere su solución.

Dedicar más o menos invocaciones a la búsqueda de los mejores parámetros de búsqueda previa a la optimización de la función es decisión vuestra. Preguntad de nuevo a la IA por este reparto.